

SECURE ACADEMIC CERTIFICATE VERIFICATION SYSTEM USING BLOCKCHAIN CONCEPTS AND SHA-256 HASHING

A PROJECT REPORT

Submitted by

LOGESHWARAN B	611223114050
SRIHARI M	611223114099
PRASANNA J	611223114071
DHAMODHARAN A	611223114018

*In partial fulfillment for the award of the
degree of*

BACHELOR OF ENGINEERING

IN

MECHANICAL ENGINEERING

**KNOWLEDGE INSTITUTE OF TECHNOLOGY
(KIOT Campus, Kakapalayam, Salem – 637504)**

**ANNA UNIVERSITY:CHENNAI 600 025
JUL 2026**

**KNOWLEDGE INSTITUTE OF TECHNOLOGY
(KIOT Campus, Kakapalayam, Salem – 637504)**

**ANNA UNIVERSITY:CHENNAI 600 025
JUL 2026**

BONAFIDE CERTIFICATE

Certified that this project report titled "**secure academic certificate verification system using block chain concepts and sha-256 hashing**"
Is the bonafide work of

LOGESHWARAN B	611223114050
SRIHARI M	611223114099
PRASANNA J	611223114071
DHAMODHARAN A	611223114018

who carried out the project work under my supervision in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering in Mechanical Engineering** of Anna University, Chennai during the academic year **2026–2027**.

SIGNATURE

DR. K.S. PRABHAKARAN, M.E., PH.D.
HEAD OF THE DEPARTMENT
DEPARTMENT OF MECHANICAL
ENGINEERING
KNOWLEDGE INSTITUTE OF
TECHNOLOGY
KIOT CAMPUS, KAKAPALAYAM (P.O.)
SALEM – 637504

SIGNATURE

**PROF. A. GAJENDRAN, B.E., M.E.,
PH.D.**
SUPERVISOR
ASSISTANT PROFESSOR
DEPARTMENT OF MECHANICAL
ENGINEERING
KNOWLEDGE INSTITUTE OF
TECHNOLOGY
KIOT CAMPUS, KAKAPALAYAM (P.O.)
SALEM – 637504

ABSTRACT

The need for verifying digital academic certificates has become crucial for educational institutions, employers, and government agencies, since verifying certificates is a time-consuming process. Manual certificate verification is more prone to errors and certificate forgery, which allows individuals to secure unauthorized employment, thereby hampering the credibility of institutes. This highlights the need for a secure and reliable certificate verification system.

This project focuses on the development of a Secure Academic Certificate Verification System using the concepts of blockchain and SHA-256 Hashing. The proposed system aims to provide a seamless and foolproof way of issuing and verifying digital academic certificates. The system is designed using Python, Flask, SQLite, HTML, and CSS to create a Model-View-Controller web application for issuing, managing and verifying certificates using a web interface.

Instead of relying on a public blockchain system, the system links certificate records using cryptographic hashing, making it possible to build a reliable and tamper-proof certificate verification system. On issuance of a certificate, a unique SHA-256 hash is generated using the uploaded certificate file for authenticity control. The certificate is added to the blockchain as a block comprising the certificate content, timestamp, previous block hash, and the current hash forming a hash chain. Additionally, the system generates a QR code for each certificate for easy access and scanning to verify the certificate. Upon verification, the hash of the uploaded certificate is generated and compared with the stored hash value. If both hash values match, it implies that the certificate is valid; otherwise, it is rejected due to tampering or being a counterfeit certificate.

The system also features a certificate search engine, blockchain explorer, certificate download engine, revoke certificate module to revoke issued certificates while maintaining the integrity of the blockchain database, and other related features. In summary, the proposed system offers a reliable, cost-effective, and efficient way of verifying academic certificates while eliminating the need for manual and error-prone processes and the risk of certificate forgery. The system also provides an insight into how the concepts of blockchain and hashing can be leveraged to develop secure and reliable applications for certificate verification.

TABLE OF CONTENT

CHAPTER NO	PAGE NO
ABSTRACT	ii
LIST OF FIGURES	v
LIST OF TABLES	vi
1. INTRODUCTION	1
1.1 Problem Statement	2
1.2 Objectives	2
1.3 Methodology	3
2. LITERATURE REVIEW	5
3. SYSTEM ANALYSIS	7
3.1 Existing System	7
3.2 Proposed system	8
3.3 Advantages of Proposed System	8
3.4 Functional Requirements	9
3.5 Non-Functional Requirements	10
4. SYSTEM DESIGN	11
4.1 System Architecture	11
4.2 Project Workflow	12
4.3 Use Case Diagram	12
4.4 Activity Diagram	14
4.5 Sequence Diagram	14
4.6 Database Design	16
5. SYSTEM IMPLEMENTATION	17
5.1 Software Requirements	17
5.2 Admin Login Module	18
5.3 Certificate Upload Module	19
5.4 SHA-256 Hash Verification	19
5.5 Blockchain Record Creation	20
5.6 QR Code Generation	20
5.7 Certificate Verification	21
5.8 Blockchain Explorer	21
5.9 Certificate Search	22
5.10 Certificate Download	22

5.11 Certificate Revocation	23
6. RESULTS AND DISCUSSION	24
6.1 Home Page	24
6.2 Admin Login	25
6.3 Dashboard	25
6.4 Certificate Upload	26
6.5 Certificate Verification	27
6.6 Blockchain Explorer	27
6.7 Search Results	28
6.8 QR Code Verification	29
6.9 Testing and Validation	30
6.10 Discussion	32
7. CONCLUSION	33
8. APPENDICES	34
8.1 Pseudocode	34
8.2 HTML Templates	40
8.3 CSS Stylesheet	40
8.4 Database Schema	41
9. REFERENCES	42

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 1.1	Working Methodology	4
Figure 4.1	System Architecture Diagram	11
Figure 4.2	Project Workflow	12
Figure 4.3	Use Case Diagram	13
Figure 4.4	Activity Diagram	14
Figure 4.5	Sequence Diagram	15
Figure 6.1	Home Page	24
Figure 6.2	Admin Login	25
Figure 6.3	Admin Dashboard	26
Figure 6.4	Certificate Upload	26
Figure 6.5	Certificate Verification	27
Figure 6.6	Blockchain Explorer	28
Figure 6.7	Certificate Search	29
Figure 6.8	QR Code Verification	29

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 4.1	Database Design	16
Table 5.1	Software Modules Description	17
Table 6.1	Certificate Verification Test Cases	30
Table 6.2	Test Results	31
Table 6.3	Existing System vs Proposed System	32
Table 8.1	Database Schema	41

CHAPTER 1

INTRODUCTION

The boom of modern technologies has brought the process of issuing, storing, and verifying any certificates to a new level. Many educational institutions currently use certificates as a method of obtaining higher education, employment, scholarships, and governmental verification. As the development of certificate verification technologies grew, so did the methods of forging and altering them. The traditional verification process is very tedious, time-consuming, and requires human intervention, which can lead to many mistakes. It is essential to establish secure digital verification systems to combat the growing number of altered and counterfeit certificates.

Many digital verification systems have been proposed to overcome the problems of traditional verification methods. The current state-of-the-art verification methods include advanced cryptographic algorithms, digital data storage, and internet technologies. These days, one of the most secure technologies to store and verify data is blockchain, due to its complex encryption algorithms and immutability of data. However, implementing a public blockchain is associated with high computational complexity, energy consumption, and costs, which makes it unfeasible for modest educational institutions and students.

The proposed solution, Secure Academic Certificate Verification System Using Blockchain Concepts and SHA-256 Hashing, utilizes concepts of blockchain technology to implement a certificate verification system without using a public blockchain network. The proposed solution overcomes the high computational complexity of the public blockchain by using concepts of blockchain in a single database. Thus, every time the administrator uploads a certificate, the certificate data along with the timestamp and hash values of the previous and current certificate are stored in the database. This way, the certificates are linked to each other in the blockchain, which makes any changes in the data obviously detectable.

The Secure Academic Certificate Verification System Using Blockchain Concepts and SHA-256 Hashing is built with Python programming language, Flask framework, SQLite database, HTML, and CSS. Upon uploading the certificate, the system uses the SHA-256 algorithm to obtain a hash value, which serves as a certificate's fingerprint. The system then compares the hash of the provided certificate with the hash stored in the database upon user verification. If the hashes match, then the certificate is valid; otherwise, it is not. In addition to basic certificate verification, the proposed system also implements a QR code generator to simplify the certificate verification process by scanning the QR code on the certificate. The QR code redirects

the user to a certificate verification webpage where the user can type the certificate's details to verify it. Several other features such as certificate searching, blockchain explorer, certificate downloading, and revocation are implemented in the proposed system. The certificate revocation feature allows the administrator to mark a particular certificate as revoked at a particular date, which adds to the security of the certificate verification system and ensures that the integrity of blockchain is not violated by simply deleting the certificate.

In conclusion, the proposed solution is a secure, cost-effective, and efficient certificate verification system that can be used in various educational institutions.

1.1 Problem Statement

Education institutions have traditionally dealt with certificate verification in a very conventional manner. It involves human efforts in checking the contents of the student's record and organization's data base. This is a very time-consuming process and requires a lot of resources. It also suffers from the problems of forgery and tampering of data/documents. Employers and other organizations are often at a disadvantage if they are unable to verify a candidate's certificate efficiently and effectively. There is a need for a reliable, verifiable, and transparent approach to certificate verification that is tamper-proof and faster.

The proposed project aims to address these issues and provide a better means of verifying one's certificate. It makes use of the concepts of blockchain, SHA-256 hashing, and QR codes to provide an efficient and effective solution for the problem of certificate verification.

1.2 Objectives

The main goals of the project are outlined below.

- Establish a reliable web-based academic certificate verification system.
- Use a SHA-256 hashing algorithm to ensure the accuracy of each submitted certificate.
- Apply blockchain methodology to store certificate data securely in the system.
- Verify certificates by comparing hash values.
- Create QR codes to facilitate certificate verification.
- Protect certificates from forgeries and alterations.

- Offer certificate search, download, blockchain explorer, and revocation services.
- Build a reliable but affordable and convenient verification platform with open-source technologies.

1.3 Methodology

The proposed system is designed using the following steps.

- First, the administrator logs in to the system by entering the correct password. The administrator uploads the certificate information such as certificate ID, student name, register number, course, institution, issue date, and certificate file.
- The system then creates a SHA-256 hash of the certificate file uploaded. The hash is unique for every certificate uploaded into the system.
- The hash value, certificate information, timestamp, previous block hash, and the current block hash are stored in the SQLite database as a block. This way, the certificate information is stored in a blockchain network.
- At the same time, the system generates a QR code with the certificate verification information. To verify a certificate, one scans the QR code to enter the certificate verification portal. The user enters the certificate ID and uploads the certificate file to be verified. The system then generates a SHA-256 hash of the certificate file and compares it with the hash value stored in the database. If the two hash values are the same, the certificate is genuine; otherwise, it is not.
- The administrator can search certificates, download the original certificate file, view the blockchain records, and revoke certificates while keeping the blockchain records.

WORKING METHODOLOGY FLOWCHART

Secure Academic Certificate Verification System Using Blockchain Concepts and SHA-256 Hashing

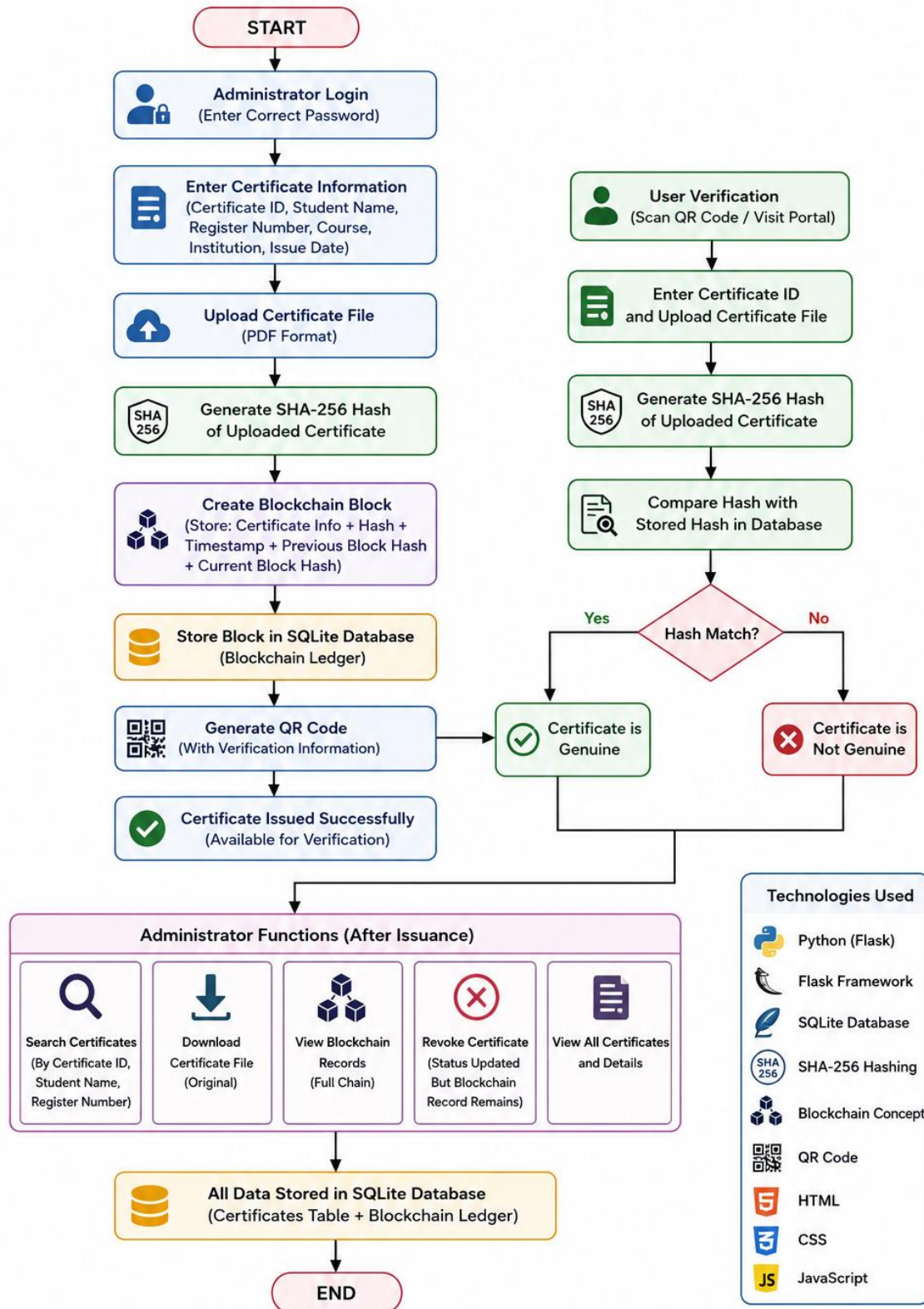


Figure 1.1 Working Methodology

CHAPTER 2

LITERATURE REVIEW

M. Sharples and J. Domingue (2016)

M. Sharples and J. Domingue (2016) suggested the application of blockchain technology for storing educational records in a convenient way. The article described the opportunity of using it for keeping students' achievements in order that they cannot be changed or falsified. The authors also discussed how this technology will guarantee the student's knowledge and provide transparency. Their main point was that blockchain can eliminate the single source of authority over the data and give students an opportunity to save their achievements in a safe place and share them without any restrictions. Therefore, this article can be used in the chosen field for suggesting ways to design certificate verification technology based on blockchain.

Dennis Yue, Joseph T. et al. (2018)

Dennis Yue with his team of researchers proposed a blockchain-based framework for issuing and verifying academic certificates in 2018. They designed a mechanism that allows storing information about the certificate and its hash in blockchain technology, thus ensuring the impossibility of forgery. The study showed that employers and institutions can quickly verify certificates without contacting the issuing department. In addition, the researchers discussed reducing the burden of administrative work and increasing the reliability of verifying academic documents. All this led to the implementation of blockchain technology in the document management system of education.

Zibin Zheng et al. (2018)

Zibin Zheng et al. (2018) provide a survey of blockchain technology that covers its architecture, consensus mechanisms, security, and applications. The paper gives an insight into the inner workings of blockchain technology, explaining how it utilizes hashing and linking of blocks to secure information and data. The authors emphasize the technology's ability to offer data protection and transparency. The study also revealed education as one of the fields that can benefit from the implementation of

blockchain technology. Specifically, the paper provides a valuable source of information for designing certificate verification based on blockchain.

Satoshi Nakamoto (2008)

Satoshi Nakamoto (2008) presented the concept of blockchain, which is applied in the Bitcoin cryptocurrency. The paper describes the process of recording transactions in blocks linked by hash functions to ensure authenticity. While the article focuses on implementing the technology in digital money, it can be used for other purposes, such as storing academic certificates. The concepts presented in the paper can be the basis of certificate verification systems since they describe the blockchain technology used nowadays to store and secure data.

NIST - Lily Chen (2015)

Lily Chen along with National Institute of Standards and Technology developed the Secure Hash Algorithm (SHA-256). The latter is considered to be an effective cryptographic hashing method due to which it becomes possible to ensure the data's integrity. According to the researchers, SHA-256 relies on a unique 256-bit hash value that makes it possible to eliminate the possibility of having two different files with the same hash. They also mentioned that any alteration of the document leads to the significant difference of its hash. In addition to the possibility for detecting unauthorized changes, this method also is applied when blockchain technologies are used, which is the main security guarantee for certificate verification applications.

CHAPTER 3

SYSTEM ANALYSIS

System analysis involves the study of the current system to establish limitations and come up with a design specification of the proposed system that satisfies demands. In the certificate verification system, the manual verification of the certificate by the authorized institution is commonly practiced. This process is often time-consuming and may involve forgeries and alterations of the information on the certificate. The proposed system aims to improve on the shortcomings of the current certificate verification system using concepts such as blockchain, SHA-256 hashing, and QR codes to ensure reliability of the certificate information. This chapter covers the background of the current certificate verification system, the proposed system, and requirements of the developed application.

3.1 Existing System

Most education systems currently utilize manual or database management approaches for verifying certificates. During verification, employers or other organizations usually get in touch with the issuing organization through emails or other formal methods of communication before verifying any certificate. This process is quite slow, since it can take several days before verification results are obtained. In addition, records of issued certificates are mostly stored in databases that can be altered by individuals who lack relevant authorization.

The current certificate verification systems do not offer tamper-proof systems, cryptography algorithms, and blockchain technology to enhance certificate verification and record management. Fake certificates in the form of image files are quite common and usually require manual verification. Additionally, current systems do not offer QR code verification or have an immutable history of all records and certificates issued.

Limitation of Existing Certificate Verification System

- Manual verification process might take a long time
- There is a high likelihood of certificate forgery
- The system is not designed to detect tampered certificates
- The system lacks transparency in managing and recording certificates
- The system also lacks blockchain technology to protect records

- The system is not easily accessible by employers or other organizations
- The current system also increases bureaucratic burden

3.2 Proposed system

The proposed system is a secure academic certificate verification system using blockchain concepts and SHA-256 hashing. It was designed using Python, Flask, SQLite, HTML, and CSS. The system offers a secure web application for issuing and verifying certificates.

Every time a certificate is uploaded into the system by an administrator, a unique SHA-256 algorithm is used to generate a summary hash value, which is saved in the database alongside the certificate details. Each issued certificate is considered a block in a blockchain-like structure that contains the certificate details, time of issue, previous block's hash, and current block's hash. To verify a certificate, the system scans a QR code that is generated for each certificate. The certificate is checked by comparing the hash of the certificate uploaded to the system against the stored hash in the database. If the hash value matches the certificate details, the certificate is authentic; otherwise, it is not. The system also allows certificate search, blockchain explorer, certificate download, and revocation features while maintaining the integrity of the blockchain records.

3.3 Advantages of Proposed System

The proposed system has several advantages over existing certificate verification mechanisms. The system uses the concept of blockchain and SHA-256 hashing algorithm to ensure certificate data is safe and secure from forgeries and alterations. The use of the web interface allows for verifying the authenticity of the certificate without requesting permission from any educational institution. In addition, the system includes QR code technology that helps provide a fast way of accessing certificate information.

Advantages

- Prevents forgery and unauthorized alteration of the certificate
- Ensures certificate verification is safe and secure using SHA-256 hashing
- Provides a blockchain concept for certificate information verification
- QR code technology facilitates fast access to certificate data
- Reduces the time of certificate verification manually

- Easy to use web interface
- Search and download certificate information
- Maintains a revocation history of certificates
- Easy and cost-effective to implement
- Improves transparency in certificate verification process

3.4 Functional Requirements

Functional requirements define the functions that the system should perform in order to accomplish its goals.

The system requires the administrator to securely log in by entering their credentials. The system should allow uploading of academic certificates including the certificate ID, student name, register number, course and issue date. Once the administrator uploads the certificate, then the system should generate a SHA-256 hash and store it in the linked list after linking with the previous block hash. The system should also automatically generate a QR code from the certificate. The system should verify certificates authenticity by comparing the certificate hash and its hash value. The system should allow searching of records in the certificate records database. The system should allow administrator to download the certificate. The system should also allow revoking the certificate without deleting the records from the blockchain. The system should display the blocks explorer and check the integrity of the blocks.

Functional Requirements

- Secure administrator login
- Upload academic certificates
- Generate SHA-256 hash
- Create linked list records
- Generate QR code
- Verify certificate authenticity
- Search certificate records
- Download certificates
- Revoke certificates
- Display blocks explorer
- Perform blockchain integrity checking.

3.5 Non-Functional Requirements

Non-functional requirements specify the quality attributes and system features that the proposed application must provide. First, the application should create a safe area for the certificate data stored in it, protect this data from external interference, and ensure the integrity of certificate information. Second, the tool should quickly provide the verification result to the user, thus reducing the response time. Third, the application's web interface should be intuitive and easy to use, even for people who are unfamiliar with the technical details of the tool.

Finally, the tool should be designed in such a way that it is reliable and always available to perform its functions, such as calculating and verifying hashes. Moreover, it should be easy to maintain, scalable, and able to provide additional services like cloud storage, digital signing, and multi-user authorization. Since the proposed solution is built using Python, Flask, SQLite, HTML, and CSS, it is inexpensive to develop and can be hosted on conventional computer hardware. Thus, the application will be affordable and accessible to the target audience.

Non-Functional Requirements

- Highly secure.
- Quick verification.
- Easy-to-use interface.
- Always available.
- Certificate information integrity.
- Easy maintenance.
- Designed for further development.
- Low cost.
- Modern web browser compatibility.

CHAPTER 4

SYSTEM DESIGN

The design of the system under consideration involves the following stages: upload of the certificate, generation of a SHA-256 hash, storing data on the blockchain, creating a QR code for storing information, certificate verification, searching, downloading, revoking, and managing the database.

4.1 System Architecture

The proposed system consists of three main layers:

Presentation Layer:

The user interface is developed using HTML and CSS. It includes the login page, dashboard, certificate upload page, verification page, search page, and blockchain explorer.

Application Layer:

Python and Flask process user requests, generate SHA-256 hashes, create blockchain records, generate QR codes, and verify certificates.

Database Layer:

SQLite stores certificate details, file hashes, previous hashes, block hashes, QR code paths, timestamps, and certificate status.

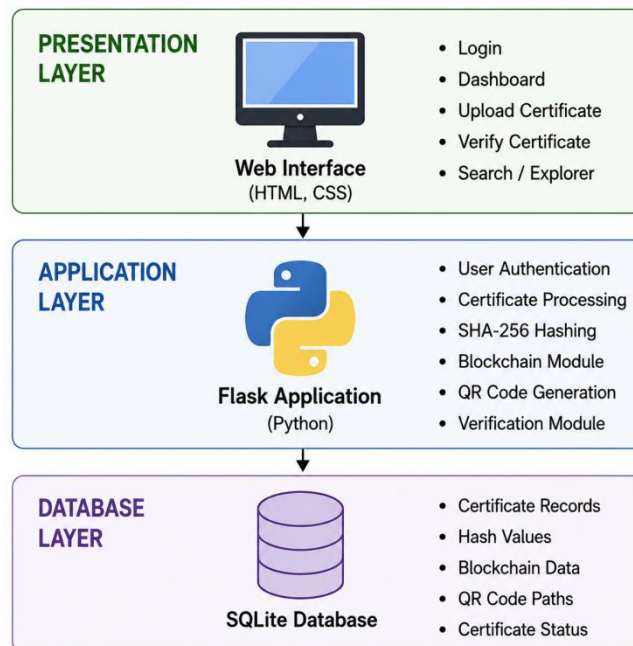


Figure 4.1 System Architecture Diagram

Figure 4.1-System Architecture Diagram

4.2 Project Workflow

The administrator logs in and uploads a certificate with all the student's data. The system calculates the SHA-256 hash, gets the previous block's hash, makes the current block's hash, saves the record in the SQLite database, and creates a QR code. On the verification stage, a user inputs the certificate's ID and uploads the file. The system cross-checks the hashes and outputs the certificate's validity – valid or invalid, revoked or not.

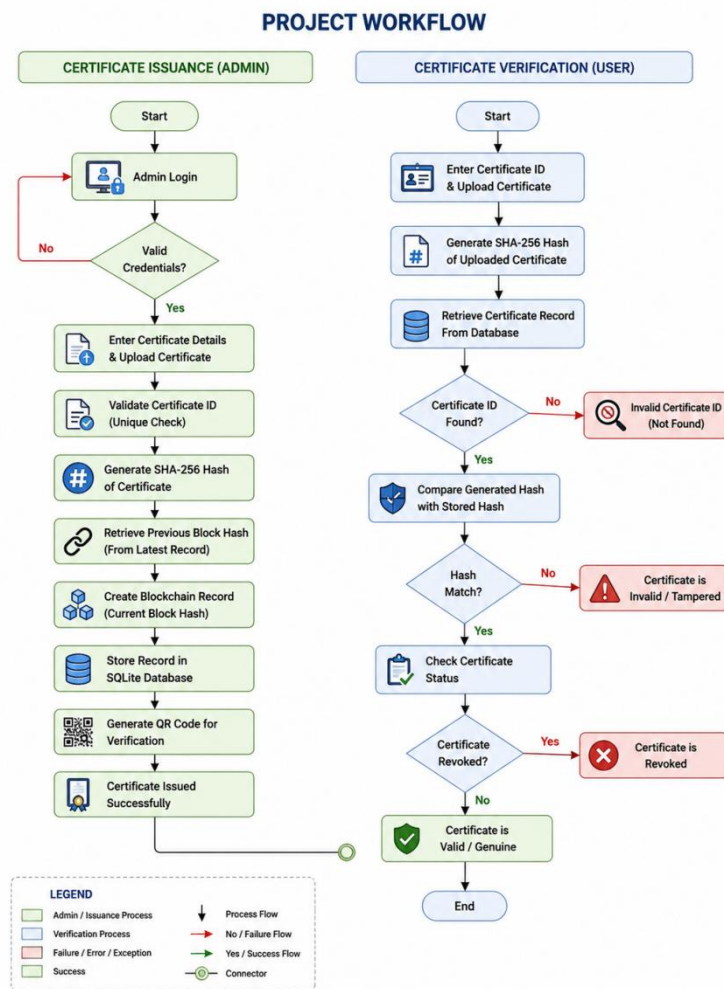


Figure 4.2 Project Workflow

Figure 4.2-Project Workflow

4.3 Use Case Diagram

The system has two types of users, administrator and certificate verifier. The administrator's dashboard allows him to login to the system, upload certificate details, search certificate, download certificate, revoke certificate, view on the blockchain explorer, and check the integrity of the certificate. On the other hand, the

verifier dashboard lets users enter certificate id, upload certificate, scan QR code, and view verification result.

Administrator Verifier

- Login
- Upload certificate
- Search certificate
- Download certificate
- Revoke certificate
- View blockchain explorer
- Check blockchain integrity

Verifier functions

- Enter certificate ID
- Upload certificate
- Scan QR code
- View verification result

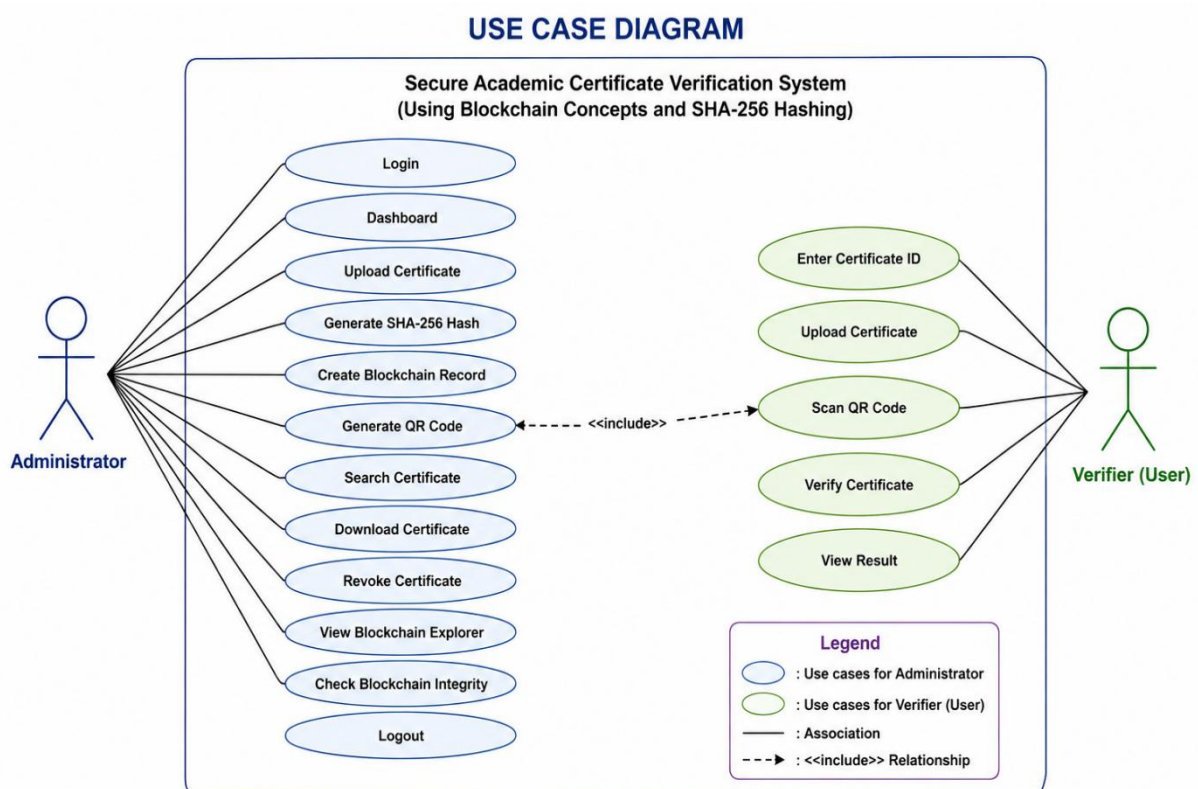


Figure 4.3 Use Case Diagram

4.4 Activity Diagram

The process begins when a certificate owner logs into the system. After logging in, the user uploads the certificate and provides the necessary information. Then, the system creates a certificate hash and blockchain block, saves the record, and generates a QR-code.

The verification process starts when an administrator scans the QR-code. The system finds the certificate record, creates a hash of the uploaded file and compares it with the stored one, checks the certificate status, and displays the result.

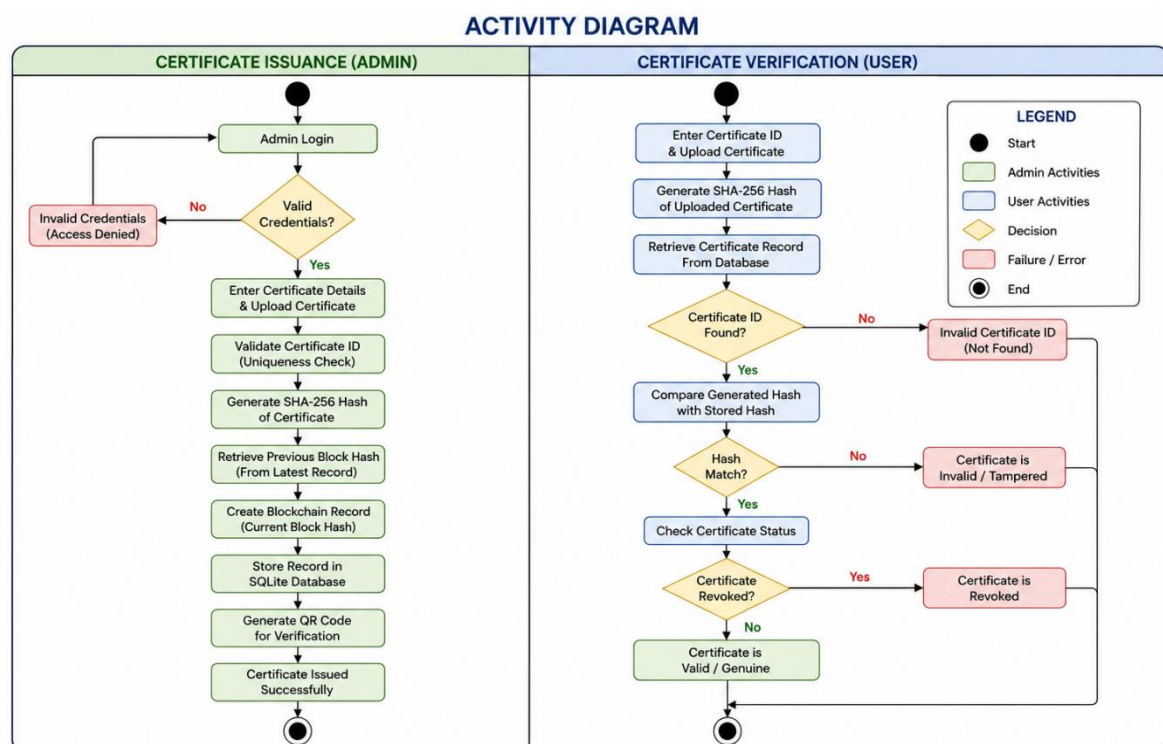


Figure 4.4 Activity Diagram

Figure 4.4 Activity Diagram

4.5 Sequence Diagram

The web interface sends the certificate information to the Flask server during the upload process. The server calculates the SHA-256 hash value, extracts the

previous hash, constructs the block hash, saves the record in the SQLite database, and generates the QR code.

The Flask server retrieves the certificate record from the SQLite database during the verification stage. It also calculates the hash of the provided certificate and compares it to the one stored in the database to verify its status and display the result.

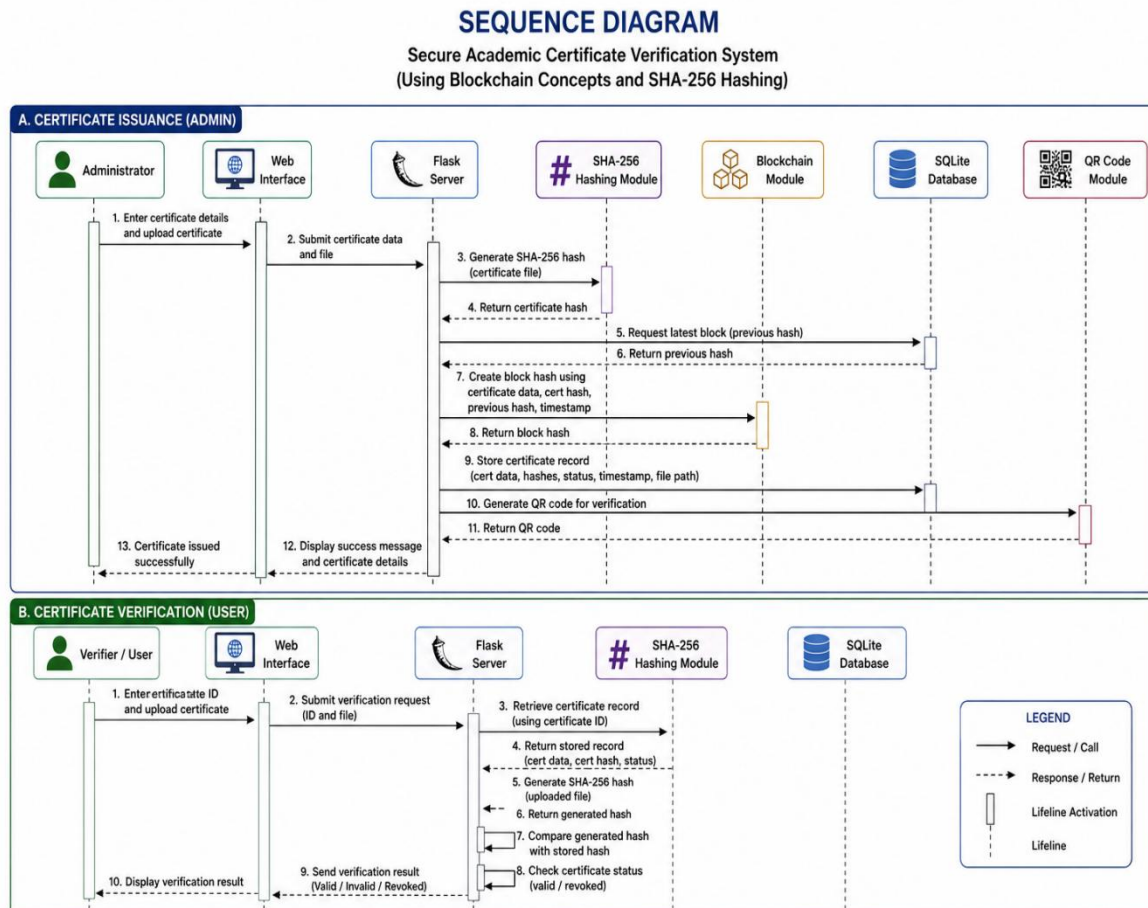


Figure 4.5 Sequence Diagram

4.6 Database Design

The SQLite database stores all certificate and blockchain information. Each certificate is treated as one blockchain block.

Field Name	Description
Certificate ID	Unique certificate number
Student Name	Name of the student
Register Number	Student register number
Course	Name of the course
Issue Date	Certificate issue date
Certificate Hash	SHA-256 hash of the file
Previous Hash	Hash of the previous block
Block Hash	Hash of the current block
QR Code Path	Location of generated QR code
Timestamp	Record creation time
Status	Valid or revoked

Table 4.1 Database Design

CHAPTER 5

SYSTEM IMPLEMENTATION

The system implementation section describes how the proposed certificate verification application was designed and how each of its modules operates. The certificate verification application was designed and developed as a web application using python framework flask. HTML and CSS were used to design the user interface while SQLite was used as the database to store the certificate and blockchain records.

The system consists of modules including administrator authentication, certificate upload, generation of hash values using SHA-256, blockchain record generation, QR code generation, certificate verification, searching, downloading, revocation, and blockchain exploration. All these modules were integrated into a single application that offers academic certificate management solutions.

5.1 Software Requirements

The software requirements specify the technologies and tools used to develop and execute the application.

Software/Technology	Purpose
Windows 11	Operating System
Visual Studio Code	Development Environment (Code Editor)
Python 3.x	Backend Programming Language
Flask	Web Application Framework
SQLite	Database Management System
HTML5	Frontend Structure
CSS3	Frontend Styling
JavaScript	Client-side Functions
SHA-256	Cryptographic Hash Algorithm
QR Code Library	QR Code Generation
Google Chrome / Microsoft Edge	Web Browser for Testing

Table 5.1 Software Modules Description

The proposed system was designed using the Visual Studio Code (VS Code) as the integrated development environment (IDE). This platform enabled effective writing, editing, debugging, and management of the project's source code. Additionally, the application's backend was designed using the Python programming language and the Flask framework. Concurrently, the user interface (UI) of the project was built using Hypertext Markup Language (HTML), Cascading Style Sheets (CSS), and JavaScript programming language. Moreover, the SQLite database was utilized to store the certificate records and blockchain data. Finally, the Secure Hash Algorithm 256 (SHA-256) and QR Code libraries were integrated into the system for cryptographic hashing and certificate QR code generation, respectively.

5.2 Admin Login Module

The admin login module grants access to the certificate management system. The system requires the administrator to enter the username and password. The Flask application verifies the input data and grants access if the credentials are correct. Upon successful login, the administrator is redirected to the dashboard. Incorrect credentials prompt an error message, and the flask application denies access. The flask application also employs session management to prevent unauthorized access to the dashboard by ensuring that the user is redirected away from the dashboard upon logging out.

The admin dashboard enables the administrator to perform the following tasks:

- Upload certificates
- Search certificates
- Download certificate files
- Revoke certificates
- View blockchain records
- Check blockchain integrity
- Logout

The login module thus ensures confidentiality of the certificate management system by allowing only authorized users to perform specific functions.

5.3 Certificate Upload Module

The certificate upload module enables the administrator to issue and store new academic certificates. In this process, the administrator enters information such as certificate id, student name, register number, course, institution, issue date, and certificate file. Firstly, the system checks whether all fields are filled before storing the certificate. Secondly, it also ensures that the certificate id entered by the administrator does not already exist in the database. If the certificate id already exists, the system will decline uploading of the certificate to avoid storing duplicate information.

After successful validation of the certificate details, the certificate file is stored in an upload folder. Thereafter, the certificate details are transferred to the hashing module and blockchain module. Upon successful completion of all these processes, the system shows a pop-up message indicating successful certificate upload. Thus, the certificate upload module ensures that all the information about the certificate is complete and correct.

5.4 SHA-256 Hash Verification

SHA-256 is the hashing algorithm that is used to create a digital fingerprint of each uploaded certificate. In particular, when the administrator uploads a certificate, it is read in binary format and processed using the SHA-256 algorithm.

As a result, a hash value consisting of 64 hexadecimal characters is obtained. At the same time, changing any certificate attribute (for example, changing any letter or picture) will lead to the fact that the hash value of this certificate will be different from the original one.

The hash of the certificate is stored in the SQLite database. Simultaneously, a similar hash is generated from the uploaded file and compared with the stored one. If the values match, the certificate is considered valid; otherwise, it is rejected. Thus, using the SHA-256 algorithm, certificate forgery can be prevented.

5.5 Blockchain Record Creation

The blockchain record creation module stores certificate records through cryptographic hashing. Each certificate is treated as a block of a blockchain-like structure.

Each block contains the following data:

- The data of the certificate itself
- Certificate Hash
- Previous block (if any)
- Current block
- The timestamp
- Certificate Status

For the initial block, the hash of the previous block is set to zero or another predetermined value. For each subsequent block, the hash of the last block is used as the hash of the previous block.

The hash of the current block is computed from the certificate data and certificate hash, the hash of the previous block, and the timestamp. This makes the blockchain record of certificate blocks immutable because modifying any block would change the block hash and break the link to the next block. Additionally, such tampering would be detected by the blockchain integrity check.

5.6 QR Code Generation

The QR code generation module creates a unique QR code for each certificate. Once the certificate is uploaded and stored in the application, a QR code containing the certificate verification information is created.

The QR code can contain the certificate ID or a link to the certificate verification page. The QR code is then saved in a separate QR code folder, and the path to this QR code is stored in the database.

The QR code can be scanned with a smartphone to quickly access certificate verification information. This eliminates the need to manually enter certificate details. In addition, QR codes provide a convenient way to verify certificates. However, it should be noted that QR codes are not a replacement for cryptographic verification. The QR code only provides a faster way to access the certificate verification system, but the certificate's authenticity is still ensured by the SHA-256 hash.

5.7 Certificate Verification

The certificate verification module is the primary function of the proposed system. By the given certificate ID and uploading the certificate file, the user can verify the authenticity of the certificate.

First, the system searches the database for a certificate record by the given certificate ID.

If the certificate ID is not found, the system displays an invalid certificate notification. Next, if the certificate ID is found, the system calculates the SHA-256 hash value of the given certificate file and compares it with the certificate hash value stored in the database.

The result of the certificate verification is determined based on the following cases:

- **Valid certificate:** the certificate ID exists in the database and matches the given certificate file hash value, and the certificate status is valid.
- **Invalid certificate:** the certificate ID exists in the database but does not match the given certificate file hash value.
- **Revoked certificate:** the certificate ID exists in the database and matches the given certificate file hash value, but the certificate status is revoked.
- **Not found:** the certificate ID does not exist in the database.

This certificate verification module allows fast and reliable verification of the authenticity of the certificate without contacting the issuing institution.

5.8 Blockchain Explorer

The blockchain explorer below demonstrates all the certificate blocks saved in the system. It is possible to check how the certificates are connected to form a blockchain by using the explorer. For example, one can view the certificate details using the given block number. The following details below describe the content of each block:

- Block number
- Certificate ID
- Student name
- Certificate hash
- Previous hash
- Current hash
- Timestamp

➤ Certificate Status

It is also possible to check whether the previous hash of a certificate block matches the current hash of the previous certificate block using the explorer. The system has a blockchain integrity checking feature that helps monitor the system's performance. It determines whether the blockchain is valid or not by recomputing the hash of all the blocks. The validity can also be checked by comparing how the records are connected. If there is no difference, then the system outputs that the blockchain is valid. On the other hand, if there are inconsistencies in the records or connection, it outputs that the system is not valid.

5.9 Certificate Search

The certificate search module is designed to enable the administrator to search for certificate records with ease. The search can be done using certificate ID, student name, registration number, or course.

The certificate search engine works by taking the query from the flask application and searching through the Sqlite database to find matches. The search results are then displayed on the results page with their vital information and status.

The certificate search module is essential for fast retrieval of certificate records from the database, especially when there are multiple records. The search feature also gives an option to download and revoke certificates directly.

5.10 Certificate Download

The certificate download module allows the administrator to download the original certificate file.

The certificate records contain the file name and path where the certificate is uploaded. When the download option is selected, the flask application gets the respective file and provides it to the browser as a response.

This way, the organizations can have easy access to their issued certificates without going through their local system directories to find them. The certificate download functionality is only available to the administrator to avoid unauthorized access to this sensitive information.

5.11 Certificate Revocation

The certificate revocation module is utilized when a certificate issued to a student needs to be invalidated. Some of the reasons for this include issuance error, cancellation of qualifications, disciplinary actions, and/or incorrect student details. Rather than deleting the certificate, the revocation feature is used to indicate that the certificate is no longer valid. The date and time of revocation are also stored in the database.

Deletion of any certificate record is not an option, since it would compromise the blockchain data. However, revoking a certificate is a way of showing that it is invalid, while still leaving the blockchain record intact, but with a status change.

When a revoked certificate is verified, it is indicated so with a warning message. Once revoked, the certificate cannot be revoked again, since the ‘revoke’ button becomes inactive and a message “Already Revoked” appears.

CHAPTER 6

RESULTS AND DISCUSSION

The designed secure academic certificate verification system was implemented using the concepts of blockchain technology and SHA-256 hashing algorithm. The developed prototype application was built using Visual Studio Code, Python, Flask, and SQLite. The designed prototype was tested and found to be effective in issuing SHA-256 hashed certificate records, blockchain generation, QR code generation, and certificate verification. The results illustrate that the developed prototype was successful in achieving its intended purpose securely and efficiently.

6.1 Home Page

The home page is the place where the user will be landed for the first time on the application. Here, one can find a lot of buttons for using the certificate verification service and log in as an administrator. The interface is straightforward and intuitive enough, so the user will not be lost on the homepage.

The home page is the central interface, providing users with access to all the available functions of the application and convenient access to the verification page.

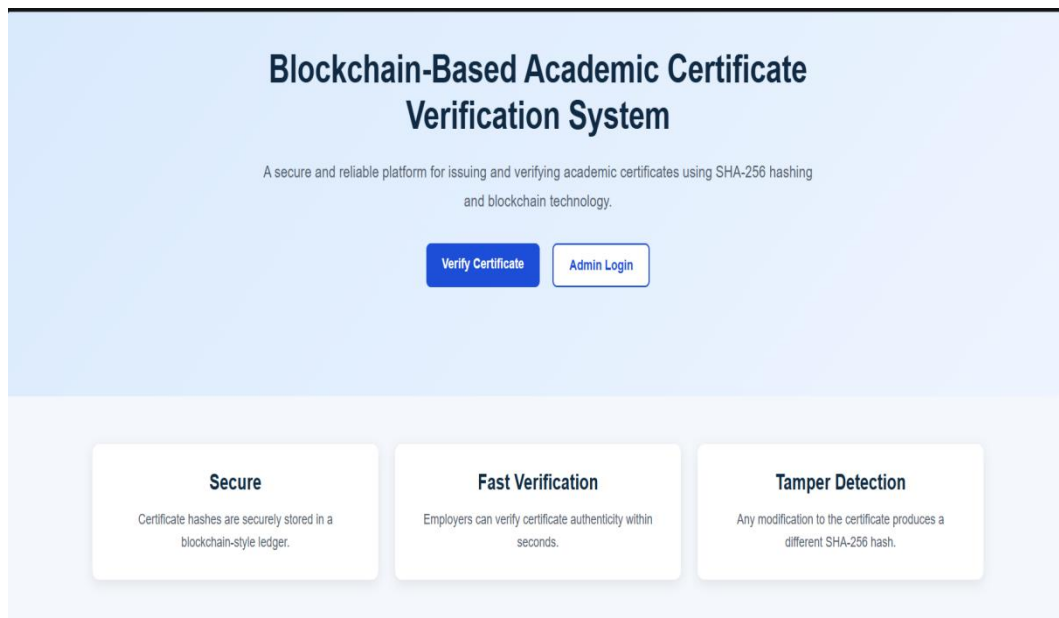


Figure 6.1 Home Page

6.2 Admin Login

The first module, i.e., the administrator login page verifies the authenticity of the user before allowing them to access the dashboard. In this process, the administrator is required to provide a valid username and password and the system automatically compares them with the stored database.

If the provided details match with those in the database, then the administrator is redirected to the dashboard. However, if they do not match, then an error message is displayed.

This process helps in ensuring that only the authorized users can view the details of certificate records.

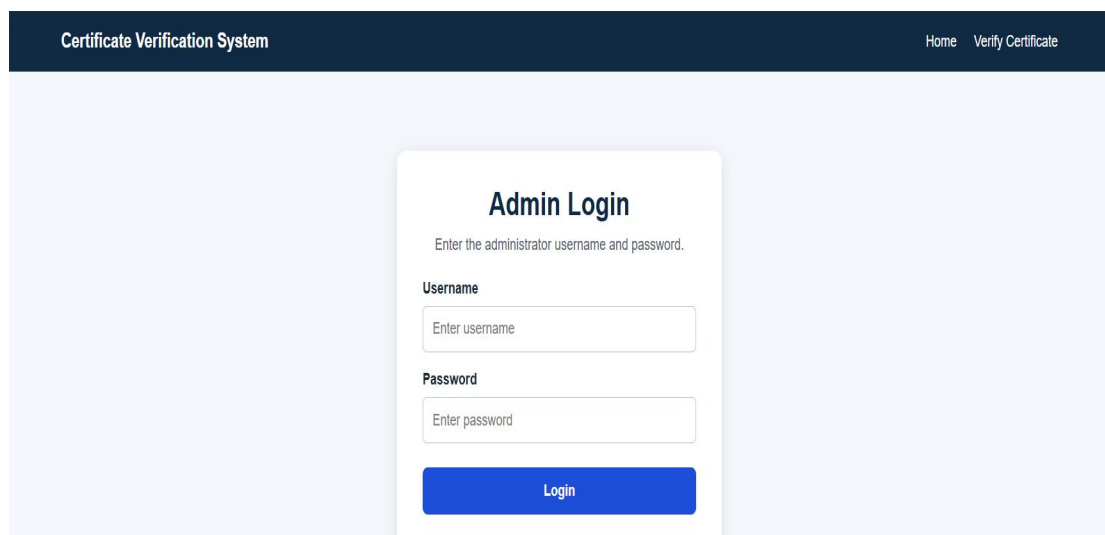
The image shows a web application interface for the 'Certificate Verification System'. At the top, there is a dark blue header bar with the system name on the left and navigation links 'Home' and 'Verify Certificate' on the right. The main content area has a light blue background. In the center, there is a white card titled 'Admin Login'. Below the title, a subtitle reads 'Enter the administrator username and password.' The card contains two input fields: 'Username' with a placeholder 'Enter username' and 'Password' with a placeholder 'Enter password'. Below these fields is a blue button labeled 'Login'.

Figure 6.2 Admin Login

6.3 Dashboard

After logging in, the administrator is welcomed by the dashboard. The dashboard contains all the available options for managing the certificates, namely: upload, search, blockchain explorer, download, revoke, and logout.

The dashboard is the main interface for managing the certificate verification system.

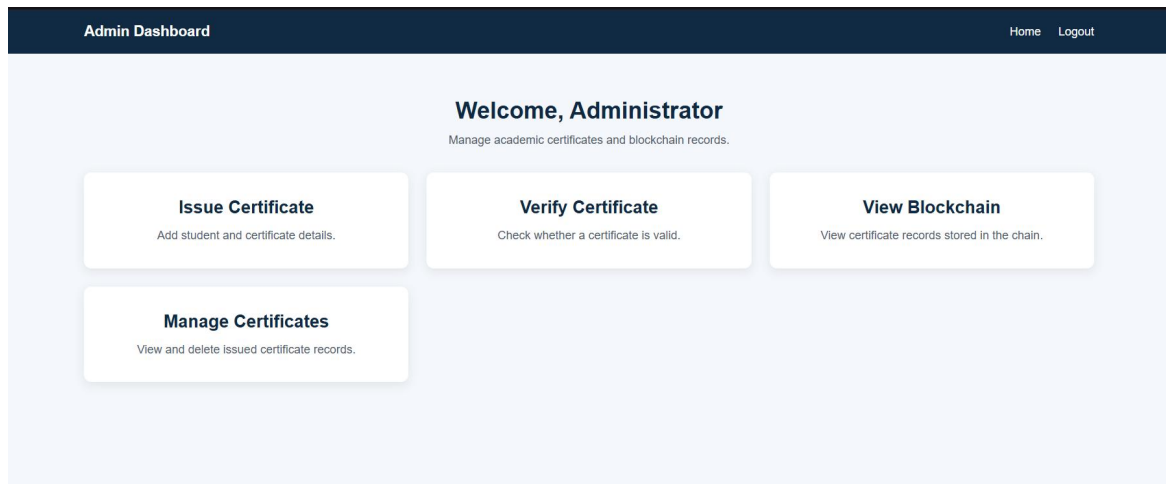


Figure 6.3 Admin Dashboard

6.4 Certificate Upload

The certificate upload module allows the administrator to input student data and upload the certificate file. After the upload, the system will:

- Generate the SHA-256 hash.
- Retrieve the previous block hash.
- Create the blockchain record.
- Store certificate details in SQLite.
- Generate the QR code.
- Display a success message saying that the certificate has been stored successfully.

The 'Issue Academic Certificate' form is displayed on a light blue background with a dark blue header containing 'Issue Academic Certificate' and 'Dashboard Logout'. The form itself is a white card with a title 'Issue Certificate' and subtitle 'Enter the academic certificate details and upload the certificate.' It contains several input fields: 'Certificate ID' (with example 'KIOT-MECH-2026-001'), 'Student Name' (placeholder 'Enter student name'), 'Register Number' (placeholder 'Enter register number'), 'Course' (with example 'B.E. Mechanical Engineering'), 'Institution' (placeholder 'Enter institution name'), and 'Issue Date' (placeholder 'dd-mm-yyyy' with a calendar icon). At the bottom, there is a 'Certificate File' section with a 'Choose File' button and 'No file chosen' text. A large blue button labeled 'Issue Certificate' is at the very bottom of the form.

Figure 6.4 Certificate Upload

6.5 Certificate Verification

The certificate verification module checks whether a certificate is genuine or not.

The user enters the Certificate ID and uploads the certificate. A new SHA-256 hash is generated and compared to the stored hash.

The verification module then displays one of the following results:

- Valid Certificate
- Invalid Certificate
- Tampered Certificate
- Revoked Certificate

This is done in an automated manner providing quick and secure verification of a certificate without human interaction.



Figure 6.5 Certificate Verification

6.6 Blockchain Explorer

The blockchain explorer visualizes every certificate as a blockchain block.

Every block consists of:

- Block Number
- Certificate ID
- Previous Hash

- Current Hash
- Timestamp
- Certificate Status

The explorer enables the administrator to ensure the integrity of the certificate blockchain and ensures that all blocks are correctly linked.

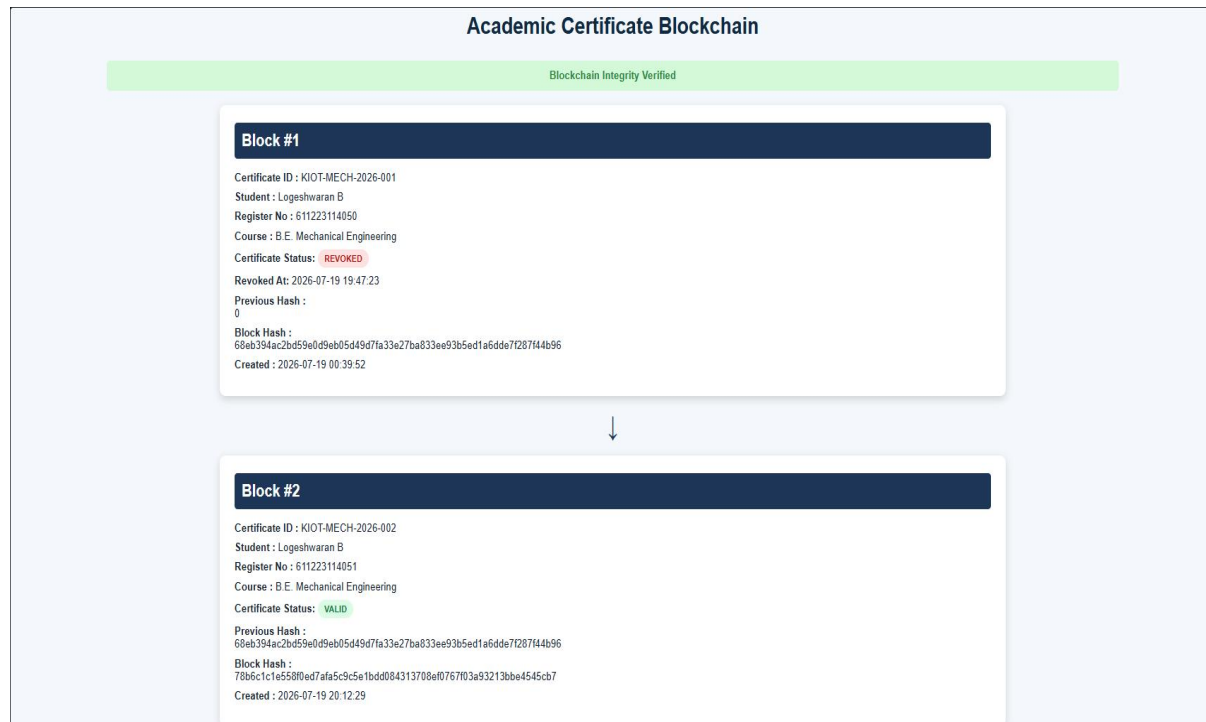


Figure 6.6 Blockchain Explorer

6.7 Search Results

The search module is used to locate the certificate by the administrator using either the certificate ID or the student's information.

The matching results appear with their verification status on the screen.

The search function enhances operability by reducing the time taken to locate a certificate record.

Manage Certificates							
Issued Certificate Records							
Search Certificate Search							
ID	Certificate ID	Student	Register Number	Course	Issue Date	Status	Action
1	KIOT-MECH-2026-001	Logeshwaran B	611223114050	B.E. Mechanical Engineering	2005-11-15	REVOKED	Already Revoked
2	KIOT-MECH-2026-002	Logeshwaran B	611223114051	B.E. Mechanical Engineering	2005-11-15	VALID	Revoke
3	KIOT-MECH-2026-050	Logeshwaran B	611223114050	B.E. Mechanical Engineering	2005-11-15	VALID	Revoke

Figure 6.7 Certificate Search

6.8 QR Code Verification

Every certificate is issued with its own QR code.

Once one scans the QR code one gets directed to the certificate verification page where one can verify the authenticity of the certificate. This QR code makes it easier and faster for one to check on the certificate.

The QR code reduces the amount of data entry when verifying the certificate.

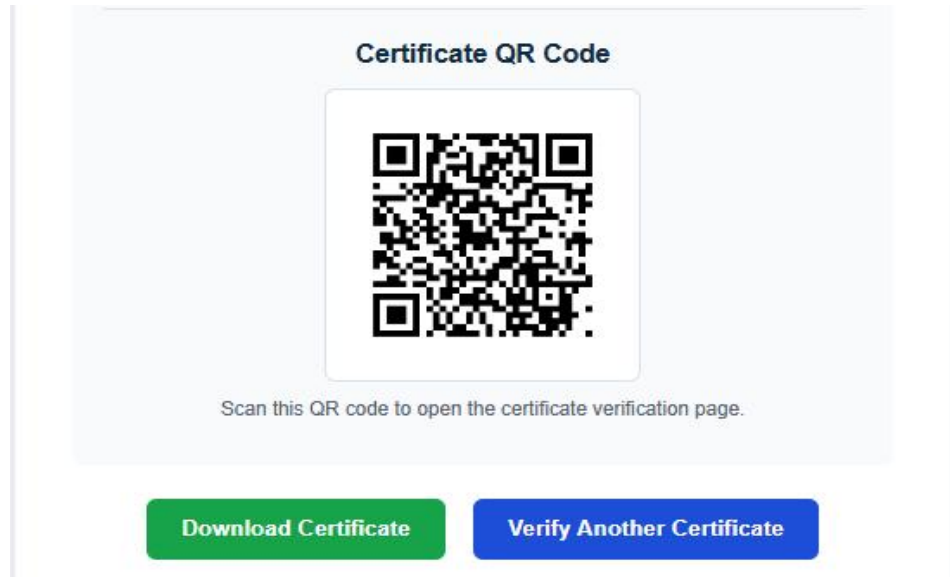


Figure 6.8 QR Code Verification

6.9 Testing and Validation

The developed application was tested by performing a set of test cases that checked the correctness of each module. The testing process proved that each module performed its functions correctly and met the requirements. Thus, it was found that the proposed certificate verification system works correctly and securely.

Test Case ID	Test Scenario	Input	Expected Result	Status
TC-01	Admin Login with valid credentials	Correct username & password	Login successful and dashboard displayed	Pass
TC-02	Admin Login with invalid credentials	Incorrect username/password	Error message displayed	Pass
TC-03	Upload certificate	Valid certificate details and PDF	Certificate stored successfully	Pass
TC-04	Generate SHA-256 hash	Upload certificate file	Unique hash generated	Pass
TC-05	Create blockchain block	Certificate uploaded	Block created with previous hash and current hash	Pass
TC-06	Generate QR Code	Certificate uploaded	QR code generated successfully	Pass
TC-07	Verify valid certificate	Correct Certificate ID and original file	Certificate verified successfully	Pass
TC-08	Verify tampered certificate	Modified certificate file	Certificate verification failed	Pass
TC-09	Search certificate	Enter Certificate ID	Matching certificate displayed	Pass
TC-10	Revoke certificate	Click Revoke button	Certificate status changed to Revoked	Pass

Table 6.1 Certificate Verification Test Cases

Table 6.1 presents the functional test cases used to validate the developed system. Each module was tested with valid and invalid inputs, and all the modules produced the expected outputs successfully.

Module	Testing Performed	Expected Output	Actual Output	Result
Home Page	Open application	Homepage displayed	Homepage displayed	Pass
Admin Login	Login with valid credentials	Dashboard opens	Dashboard opened	Pass
Certificate Upload	Upload certificate	Certificate saved	Successfully uploaded	Pass
SHA-256 Hashing	Generate hash	Unique hash value	Hash generated correctly	Pass
Blockchain Creation	Store blockchain block	Block added	Block stored successfully	Pass
QR Code Generation	Generate QR code	QR image created	QR generated successfully	Pass
Certificate Verification	Verify original certificate	Valid certificate displayed	Verification successful	Pass
Certificate Search	Search by Certificate ID	Certificate found	Correct record displayed	Pass
Download Certificate	Download certificate	PDF downloaded	Download successful	Pass
Blockchain Explorer	View blockchain	All blocks displayed	Blockchain displayed correctly	Pass
Certificate Revocation	Revoke certificate	Status changed to Revoked	Revocation successful	Pass

Table 6.2 Test Results

Table 6.2 shows the testing results of all the developed modules. Every function including certificate upload, blockchain generation, QR code generation, verification, search, download, and revocation was executed successfully.

Feature	Existing System	Proposed System
Certificate Storage	Stored in normal database	Stored with blockchain records
Security	Moderate	High using SHA-256 hashing
Verification	Manual verification	Instant online verification
Tamper Detection	Difficult	Automatic through hash comparison
QR Code	Not available	Generated for every certificate
Certificate Download	Limited	Available anytime
Blockchain Ledger	Not available	Blockchain-style ledger maintained
Search Facility	Limited	Quick search by Certificate ID
Revocation	Not supported	Supported with status update
Transparency	Low	High
Data Integrity	Can be modified	Protected using blockchain hashing
Authentication	Manual checking	Automated verification
Processing Time	High	Very low
Human Errors	More	Minimal
Overall Efficiency	Average	High

Table 6.3 Existing System vs Proposed System

Table 6.3 compares the existing certificate verification system with the proposed blockchain-based system. The proposed system offers enhanced security, faster verification, blockchain integrity, QR code support, certificate revocation, and improved reliability.

6.10 Discussion

The developed blockchain-based academic certificate verification system was successfully implemented and tested using Visual Studio Code, Python, Flask, SQLite, HTML, CSS, and JavaScript. The application generated SHA-256 hashes for every uploaded certificate and stored them as blockchain records. The QR code generation feature enabled quick certificate verification, while the blockchain explorer ensured transparency and data integrity. The certificate search, download, and revocation modules also worked successfully. The testing results confirmed that the proposed system provides a secure, reliable, and efficient solution for academic certificate verification compared to conventional methods.

CHAPTER 7

CONCLUSION

The Academic Certificate Verification System using Blockchain Technology was designed and developed successfully. This project was designed using Visual Studio code as an editor, Python (Flask) as a backend language, HTML, CSS, and JavaScript as frontend languages, and SQLite as a database. The developed application allows the administrator to issue academic certificates, generate a secure hash (SHA-256), add a block-chain record, and QR code for each certificate issued.

When verifying the certificate, the blockchain hash is compared to the new hash generated from the QR code scan to detect forgeries. In case of forgeries, the application detects invalid or revoked certificates, thus ensuring credibility and eliminating the manual verification of academic certificates.

This project demonstrates how the concept of blockchain was used to design an application that can be used in issuing electronic academic certificates. The application helps enhance security, reduce time taken to verify certificates manually, and prevent forgeries digitally. Besides, certificate holders can scan QR codes to confirm the authenticity and validity of their certificates. Thus, the project meets the set objectives by offering a cost-effective solution in certificate verification.

CHAPTER 8

APPENDICES

8.1 Pseudocode

Algorithm 1: Administrator Login

```
BEGIN  
  
Display Login Page  
  
Input Username and Password  
  
IF Username and Password are correct THEN  
    Open Dashboard  
ELSE  
    Display "Invalid Login Credentials"  
END IF  
  
END
```

Algorithm 2: Certificate Upload

```
BEGIN  
  
Login as Administrator  
  
Enter:  
    Certificate ID  
    Student Name  
    Register Number  
    Course  
    Institution  
    Issue Date  
  
Upload Certificate File  
  
IF File is Valid THEN  
  
    Generate SHA-256 Hash  
  
    Get Previous Block Hash
```

```
    Generate Current Block Hash  
    Generate QR Code  
    Store Certificate Details in Database  
    Display "Certificate Uploaded Successfully"  
ELSE  
    Display Error Message  
END IF  
END
```

Algorithm 3: SHA-256 Hash Generation

```
BEGIN  
Read Uploaded Certificate File  
Convert File into Binary Data  
Apply SHA-256 Algorithm  
Generate 64-character Hash Value  
Return Hash Value  
END
```

Algorithm 4: Blockchain Record Creation

BEGIN

Receive Certificate Details

Retrieve Previous Block Hash

Generate Current Timestamp

Combine:

 Certificate Details

 Certificate Hash

 Previous Hash

 Timestamp

Generate Block Hash

Store Block Information in Database

END

Algorithm 5: QR Code Generation

BEGIN

Receive Certificate ID

Create Verification URL

Generate QR Code Image

Save QR Code

Store QR Code Path in Database

END

Algorithm 6: Certificate Verification

BEGIN

Enter Certificate ID

Upload Certificate File

Search Certificate ID in Database

IF Certificate Not Found THEN

 Display "Certificate Not Found"

ELSE

 Generate SHA-256 Hash

 Compare Generated Hash with Stored Hash

 IF Hashes Match THEN

 IF Certificate Status is REVOKED THEN

 Display "Certificate Revoked"

 ELSE

 Display "Valid Certificate"

 END IF

 ELSE

 Display "Invalid Certificate"

 END IF

END IF

END

Algorithm 7: Blockchain Integrity Verification

```
BEGIN  
  
Retrieve All Blockchain Records  
  
Set Previous Hash = "0"  
  
FOR Each Block  
    Recalculate Block Hash  
    Compare Previous Hash  
    IF Hash Mismatch THEN  
        Blockchain = Invalid  
        STOP  
    END IF  
    Update Previous Hash  
END FOR  
  
Blockchain = Valid  
  
END
```

Algorithm 8: Certificate Search

```
BEGIN  
  
Enter Search Keyword  
  
Search Database Using:  
    Certificate ID  
    Student Name  
    Register Number  
  
Display Matching Records  
  
END
```

Algorithm 9: Certificate Download

```
BEGIN  
  
Select Certificate  
  
Retrieve File Path  
  
IF File Exists THEN  
    Download Certificate  
ELSE  
    Display "File Not Found"  
END IF  
  
END
```

Algorithm 10: Certificate Revocation

```
BEGIN  
  
Select Certificate  
  
Check Certificate Status  
  
IF Status = VALID THEN  
    Update Status = REVOKED  
    Save Revocation Time  
    Display "Certificate Revoked Successfully"  
ELSE  
    Display "Certificate Already Revoked"  
END IF  
  
END
```

Algorithm 11: Logout

BEGIN

Clear Administrator Session

Redirect to Home Page

END

8.2 HTML Templates

The frontend of the project was developed using HTML templates with the Jinja2 templating engine supported by Flask.

Templates Included

- index.html – Home Page
- login.html – Administrator Login
- dashboard.html – Admin Dashboard
- upload.html – Certificate Upload
- verify.html – Certificate Verification
- result.html – Verification Result
- certificates.html – Certificate Management
- blockchain.html – Blockchain Explorer

Each page provides a responsive interface for interacting with the certificate verification system.

8.3 CSS Stylesheet

The appearance of the web application is designed using a single CSS stylesheet.

File

style.css

The stylesheet provides

- Responsive page layout
- Navigation bar styling
- Dashboard cards
- Forms and input controls
- Tables

- Buttons
- Status badges
- QR Code display
- Blockchain explorer design
- Mobile-friendly interface

8.4 Database Schema

The application stores certificate information using the SQLite database.

Database Name

- database.db

Table Name

- Certificates

Fields

Field Name	Data Type	Description
id	INTEGER	Primary Key
certificate_id	TEXT	Unique Certificate ID
student_name	TEXT	Student Name
register_number	TEXT	Register Number
course	TEXT	Course Name
institution	TEXT	Institution Name
issue_date	TEXT	Certificate Issue Date
file_name	TEXT	Uploaded Certificate File
certificate_hash	TEXT	SHA-256 Hash
previous_hash	TEXT	Previous Blockchain Hash
block_hash	TEXT	Current Blockchain Hash
qr_code	TEXT	QR Code Image Path
status	TEXT	VALID / REVOKED
revoked_at	TEXT	Revocation Time
created_at	TEXT	Record Creation Time

Table 8.4 Database Schema

CHAPTER 9

REFERENCES

- 1) S. Nakamoto, Bitcoin: A Peer-to-Peer Electronic Cash System, 2008.
- 2) M. Crosby, P. Pattanayak, S. Verma, and V. Kalyanaraman, “Blockchain Technology: Beyond Bitcoin,” Applied Innovation Review, no. 2, pp. 6-19, Jun. 2016.
- 3) M. Swan, Blockchain: Blueprint for a New Economy. Sebastopol, CA, USA: O’Reilly Media, 2015.
- 4) W. Diffie and M. E. Hellman, “New Directions in Cryptography,” IEEE Transactions on Information Theory, vol. 22, no. 6, pp. 644-654, Nov. 1976.
- 5) National Institute of Standards and Technology (NIST), Secure Hash Standard (SHS), FIPS PUB 180-4, 2015.
- 6) A. Narayanan, J. Bonneau, E. Felten, A. Miller, and S. Goldfeder, Bitcoin and Cryptocurrency Technologies. Princeton University Press, 2016.
- 7) D. Yaga, P. Mell, N. Roby, and K. Scarfone, Blockchain Technology Overview, NIST Special Publication 800-193, 2018.
- 8) M. Pilkington, “Blockchain Technology: Principles and Applications,” in Research Handbook on Digital Transformations, Edward Elgar Publishing, 2016.
- 9) C. Dwork and M. Naor, “Pricing via Processing or Combatting Junk Mail,” in Advances in Cryptology – CRYPTO’92, Springer, pp. 139-147, 1993.
- 10) M. Herlihy, “Blockchains and the Future of Distributed Computing,” Communications of the ACM, vol. 62, no. 2, pp. 78-85, Feb. 2019.
- 11) A. M. Antonopoulos, Mastering Bitcoin, 2nd ed. Sebastopol, CA: O’Reilly Media, 2017.
- 12) G. Wood, Ethereum: A Secure Decentralised Generalised Transaction Ledger, Ethereum Foundation, Yellow Paper, 2014.
- 13) Hyperledger Foundation, Hyperledger Fabric Documentation, Linux Foundation, 2024.

- 14) M. Grinberg, Flask Web Development, 2nd ed. Sebastopol, CA: O'Reilly Media, 2018.
- 15) R. Hettinger, "Python Programming Language Documentation," Python Software Foundation, 2024.
- 16) SQLite Consortium, SQLite Documentation, SQLite Official Documentation, 2024.
- 17) T. Denso Wave Incorporated, QR Code Essentials and Specifications, DENSO WAVE, Japan.
- 18) R. Rivest, "The MD5 Message-Digest Algorithm," RFC 1321, Internet Engineering Task Force (IETF), Apr. 1992.
- 19) M. Bellare and P. Rogaway, "Introduction to Modern Cryptography," University of California, Davis, Lecture Notes, 2005.
- 20) T. Berners-Lee, "Information Management: A Proposal," CERN, Geneva, Switzerland, 1989.
- 21) World Wide Web Consortium (W3C), HTML5: A Vocabulary and Associated APIs for HTML and XHTML , W3C Recommendation, 2021.
- 22) World Wide Web Consortium (W3C), Cascading Style Sheets (CSS) Level 3 Specification, W3C Recommendation, 2021.
- 23) Ecma International, ECMAScript(R) 2023 Language Specification, ECMA-262, 14th Edition, 2023.
- 24) A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill Education, 2020.
- 25) I. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson Education, 2016.